

# Model Context Protocol (MCP): Conectando Contexto, Agentes e Arquitetura de Software Moderna

**Nagib Sabbag Filho**

Leaders.Tec.Br, 2(37), ISSN: 2966-263X, 2025.

e-mail: na.sabbag@gmail.com

DOI: <https://doi.org/10.5281/zenodo.18111294>

PermaLink: <https://leaders.tec.br/artigo/model-context-protocol-mcp-conectando-contexto-agentes-e-arquitetura-de-software-moderna>

Received: 26 Dec 2025 / Accepted: 28 Dec 2025 / Published online: 30 Dec 2025

---

## Abstract:

Este artigo apresenta o Model Context Protocol (MCP) como uma abordagem estruturada para integração entre agentes baseados em modelos de linguagem e sistemas corporativos, com ênfase em governança, previsibilidade e segurança. Diferentemente de integrações diretas e ad hoc, o MCP estabelece contratos explícitos para exposição de capacidades por meio de tools, resources e prompts, reduzindo ambiguidades e limitando o acesso do agente ao domínio.

## Key words:

Model Context Protocol, MCP, contexto, agentes, arquitetura de software moderna, integração, interoperabilidade, sistemas baseados em contexto, design de software, inteligência artificial, comunicação entre agentes, modelagem de contexto, desenvolvimento ágil, aplicações contextuais, escalabilidade, eficiência, automação, tecnologias emergentes, software adaptativo, interação humano-computador.

---

## Introdução

Sistemas modernos que integram agentes baseados em LLMs e assistentes inteligentes a serviços corporativos exigem mais do que simples chamadas a APIs. Esses sistemas requerem contexto explicitamente controlado, interfaces padronizadas, limites claros de acesso e mecanismos formais de governança.

A literatura científica aponta que a ausência desses elementos resulta em comportamento imprevisível, dificuldades de auditoria e riscos de segurança em sistemas baseados em IA (AMERSHI et al., 2019).

Complementando essa visão, este artigo apresenta uma implementação prática de um MCP Server em .NET operando sobre um contexto GraphQL, explicitando como definir e formalizar estruturas de contexto, limites de acesso e interfaces padronizadas. Para que modelos de linguagem (LLMs) sejam integrados de forma robusta em sistemas corporativos, com segurança, auditabilidade e previsibilidade é necessário adotar arquiteturas e mecanismos de governança que considerem restrições de acesso, rastreabilidade e mecanismos formais de controle, evitando comportamentos imprevistos ou insegurança operacional (KARRAS et al., 2025). O objetivo central é oferecer um template reproduzível para cenários reais, ilustrado por APIs que retornam resultados para um possível chat.

## Contexto do Model Context Protocol (MCP)

O MCP pode ser entendido como um contrato de integração entre um host (cliente que executa o modelo) e um server (provedor de capacidades). Em vez de o modelo “inventar” integrações, o MCP permite que o host descubra e invoque capacidades explicitamente definidas:

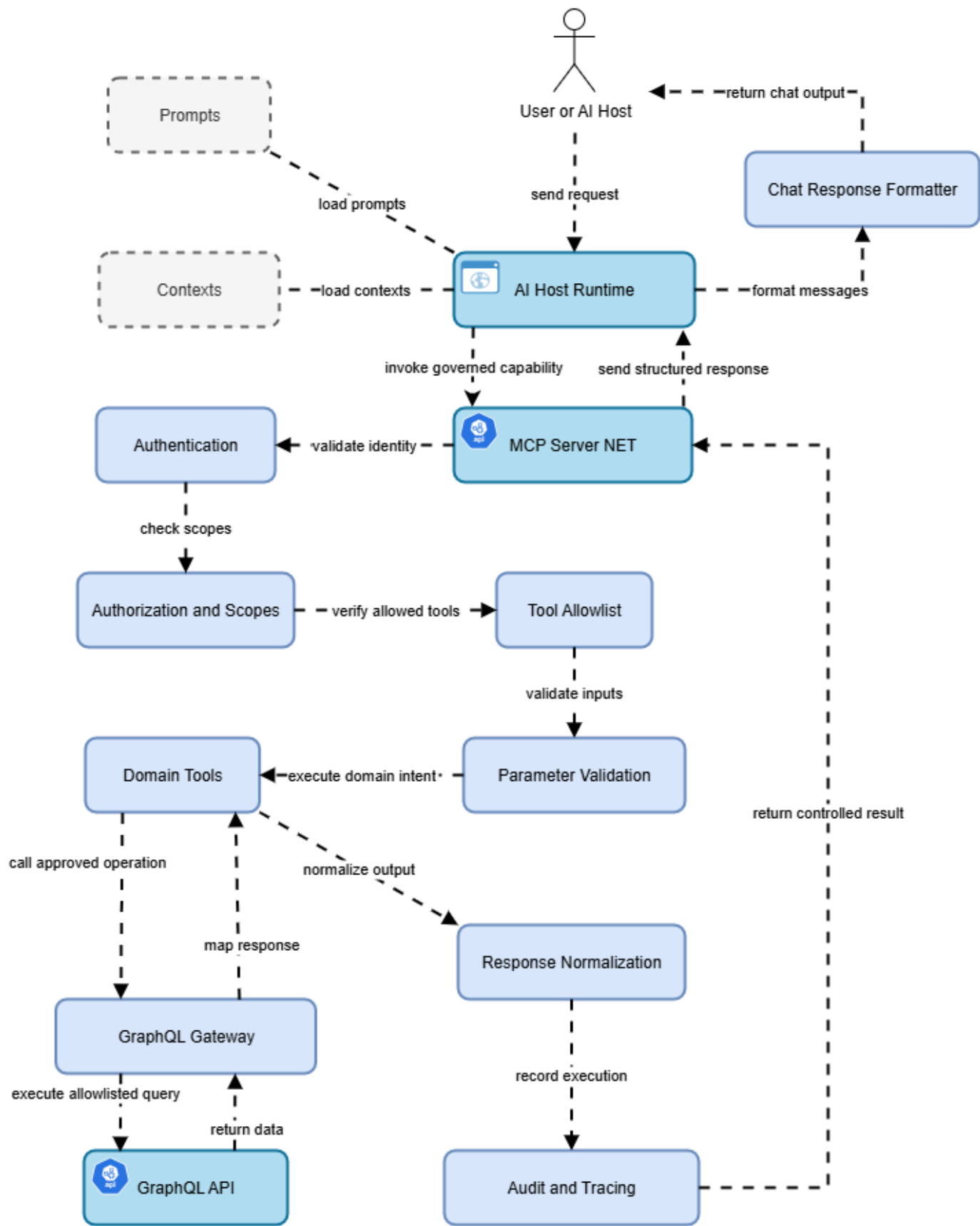
- Tools: funções invocáveis (ex.: consultar pedidos via GraphQL com allowlist).
- Resources: artefatos somente leitura que fornecem contexto (ex.: schema, políticas, exemplos).
- Prompts: templates/atalhos de interação (ex.: um comando padronizado para buscar pedidos).

Em arquiteturas modernas, esse padrão reduz acoplamento e cria um ponto único para aplicar autorização, rate limiting, auditoria e políticas de dados, evitando que o modelo tenha acesso direto e irrestrito ao backend.

### **Definição de Contexto**

Contexto, neste artigo, é o conjunto de informações e restrições que determinam o que pode ser acessado, como pode ser acessado e qual formato de resposta é aceito. No cenário proposto, o contexto é materializado por:

- Uma API GraphQL como fonte de dados do domínio.
- Um MCP Server como camada de governança, com allowlist de operações e validação de parâmetros.
- Recursos (resources) documentando schema, políticas e exemplos de saída.



Agentes e Execução Controlada

Agentes (no sentido funcional) são consumidores dessas capacidades: o host decide se e quando chamar uma tool, porém o servidor define os limites do que é permitido. Dessa forma, a execução permanece determinística do ponto de

vista de acesso: cada ação invocável possui contrato, validação e auditoria.

## Arquitetura de Software Moderna

A implementação do MCP se beneficia de princípios de microserviços e governança de APIs: responsabilidades bem definidas, camadas de segurança, observabilidade e desacoplamento. Em especial, a combinação MCP + GraphQL é útil quando se deseja flexibilidade de consulta, desde que acompanhada de limites (complexidade, profundidade e allowlist).

## Implementação do MCP em .NET sobre GraphQL

Nesta seção, apresenta-se um recorte reproduzível da arquitetura proposta, composto por:

- (i) uma API GraphQL mínima,
- (ii) um gateway com allowlist e timeout,
- (iii) tools de domínio com intenção explícita e
- (iv) uma API de demonstração capaz de retornar respostas em formato de chat.

O objetivo é evidenciar como o MCP pode operar sobre um contexto GraphQL de forma segura, auditável e previsível.

### Contexto GraphQL: API mínima (Hot Chocolate)

O contexto de domínio é exposto por meio de uma API GraphQL mínima, implementada com Hot Chocolate, fornecendo apenas os tipos e consultas estritamente necessários para o domínio de pedidos (orders). Essa abordagem reduz a superfície de ataque, facilita a governança e cria um contrato explícito de contexto para consumo por agentes e tools MCP.

O Hot Chocolate é um framework GraphQL para .NET que permite a construção de APIs fortemente tipadas, orientadas a esquema e integradas ao ecossistema ASP.NET (MARJANOVIC, 2022). Ele automatiza a geração do schema a partir de tipos C#, suporta validação, middlewares, resolução eficiente de dados e extensões como autorização e observabilidade, tornando-o adequado para expor contextos bem definidos em arquiteturas orientadas a agentes e MCPs.

```
using Microsoft.AspNetCore.Builder;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddGraphQLServer()
    .AddQueryType<Query>();

var app = builder.Build();

app.MapGraphQL("/graphql");

app.Run();

public record Order(string Id, string CustomerName, decimal Total, string Status);

public sealed class Query
{
    public IEnumerable<Order> Orders(string? status = null)
    {
        var all = new[]
        {
            new Order("ord_1001", "Ana", 120.50m, "PAID"),
            new Order("ord_1002", "Bruno", 89.90m, "PENDING"),
            new Order("ord_1003", "Carla", 450.00m, "PAID"),
        };
    }
}
```

```

return status is null
    ? all
    : all.Where(o => o.Status.Equals(status, StringComparison.OrdinalIgnoreCase));
}

public Order? OrderById(string id) =>
    Orders().FirstOrDefault(o => o.Id == id);
}

```

## Gateway GraphQL com Allowlist e Timeout

O gateway encapsula o acesso ao endpoint GraphQL e aplica políticas explícitas de controle, incluindo allowlist de operações, timeout e cancelamento. Essa camada impede a execução de consultas arbitrárias, padroniza o acesso ao contexto e reduz riscos associados a abuso, negação de serviço ou exploração do schema.

```

using System.Net.Http.Json;
using System.Text.Json;

public sealed class GraphQLGateway
{
    private readonly HttpClient _http;

    private static readonly HashSet<string> AllowedOperations =
        new(StringComparer.OrdinalIgnoreCase)
        {
            "Orders",
            "OrderById"
        };

    public GraphQLGateway(HttpClient http) => _http = http;

    public async Task<JsonElement> ExecuteAsync(
        string operationName,
        string query,
        object variables,
        CancellationToken ct)
    {
        if (!AllowedOperations.Contains(operationName))
            throw new InvalidOperationException(
                $"Operation '{operationName}' is not allowed.");

        using var cts = CancellationTokenSource.CreateLinkedTokenSource(ct);
        cts.CancelAfter(TimeSpan.FromSeconds(3));

        var payload = new
        {
            operationName,
            query,
            variables
        };

        using var resp =
            await _http.PostAsJsonAsync("/graphql", payload, cts.Token);

        resp.EnsureSuccessStatusCode();

        return await resp.Content.ReadFromJsonAsync<JsonElement>(
            cancellationToken: cts.Token);
    }
}

```

## Tools MCP: operações de domínio (sem GraphQL livre)

Em vez de expor uma tool genérica capaz de executar qualquer query GraphQL, são disponibilizadas tools orientadas a intenção, cada uma mapeando uma operação de negócio específica. Essa decisão reduz riscos, facilita auditoria, simplifica validações e reforça o princípio de menor privilégio no MCP.

```
using System.Text.Json;

public sealed class OrderTools
{
    private readonly GraphQLGateway _gql;

    public OrderTools(GraphQLGateway gql) => _gql = gql;

    // Tool: orders.search (scope sugerido: orders.search)
    public Task<JsonElement> SearchOrdersAsync(
        string? status,
        CancellationToken ct)
    {
        const string operationName = "Orders";
        const string query = """
        query Orders($status: String) {
          orders(status: $status) {
            id
            customerName
            total
            status
          }
        }
        """,
        ;

        var variables = new { status };
        return _gql.ExecuteAsync(operationName, query, variables, ct);
    }

    // Tool: orders.getByld (scope sugerido: orders.read)
    public Task<JsonElement> GetOrderByldAsync(
        string id,
        CancellationToken ct)
    {
        const string operationName = "OrderByld";
        const string query = """
        query OrderByld($id: String!) {
          orderByld(id: $id) {
            id
            customerName
            total
            status
          }
        }
        """,
        ;

        var variables = new { id };
        return _gql.ExecuteAsync(operationName, query, variables, ct);
    }
}
```

## Resources (Contexto) e Prompts (Fluxos)

Embora a implementação concreta de um MCP Server varie conforme o SDK ou framework utilizado, o conteúdo recomendado para resources e prompts permanece estável e independente de linguagem, funcionando como base de governança e alinhamento semântico entre agentes.

- Resource: Schema (resource://graphql/schema): snapshot do schema ou do subconjunto relevante (orders).
- Resource: Policies (resource://policies/access): scopes, quotas, dados proibidos e regras de auditoria.
- Resource: Examples (resource://examples/orders): exemplos de saída para padronizar o discurso e o formato.
- Prompt: /ask-orders: instrução padronizada — “use apenas orders.search e orders.getByld; não tente outras fontes; não solicite dados sensíveis”.

## API de Demonstração: Resultado em Formato de Chat

Para evidenciar o resultado sem depender de um host específico, a seguir cria-se uma API REST que retorna uma lista de mensagens (role/content). Essa API simula um fluxo: interpreta o texto do usuário e chama tools governadas.

### Contrato

- POST /chat com { "text": "..." }
- Resposta: { correlationId, messages: [{role, content}, ...] }

```
using System.Text.Json;
using Microsoft.AspNetCore.Http.HttpResults;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddHttpClient<GraphQLGateway>(http =>
{
    http.BaseAddress = new Uri("https://localhost:5001"); // GraphQL API
});

builder.Services.AddScoped<OrderTools>();

var app = builder.Build();

app.MapPost("/chat", async Task<Ok<ChatResponse>> (
    ChatRequest req,
    OrderTools tools,
    HttpContext http,
    CancellationToken ct) =>
{
    var correlationId = http.TraceIdentifier;

    var messages = new List<ChatMessage>
    {
        new("system", "Use apenas tools allowlisted. Não retorne PII. Responda de forma objetiva."),
        new("user", req.Text)
    };

    if (req.Text.Contains("pago", StringComparison.OrdinalIgnoreCase) ||
        req.Text.Contains("paid", StringComparison.OrdinalIgnoreCase))
    {
        messages.Add(new("assistant", "Consultando pedidos com status PAID via tool orders.search..."));
        var json = await tools.SearchOrdersAsync("PAID", ct);
        messages.Add(new("assistant", SummarizeOrders(json)));
        return TypedResults.Ok(new ChatResponse(correlationId, messages));
    }

    if (req.Text.Contains("ord_", StringComparison.OrdinalIgnoreCase))
```

```

{
    var id = ExtractOrderId(req.Text);
    messages.Add(new("assistant", $"Consultando o pedido {id} via tool orders.getById..."));
    var json = await tools.GetOrderByIdAsync(id, ct);
    messages.Add(new("assistant", SummarizeOrder(json, id)));
    return TypedResults.Ok(new ChatResponse(correlationId, messages));
}

messages.Add(new("assistant",
    "Exemplos: \"me traga pedidos pagos\" ou \"detalhe o pedido ord_1002\""));

return TypedResults.Ok(new ChatResponse(correlationId, messages));
});

app.Run();

static string ExtractOrderId(string text)
{
    var idx = text.IndexOf("ord_", StringComparison.OrdinalIgnoreCase);
    if (idx < 0) return "ord_1001";
    var tail = text.Substring(idx);
    var end = tail.IndexOfAny([' ', '.', ',', ';', '\n', '\r']);
    return end < 0 ? tail : tail[..end];
}

static string SummarizeOrders(JsonElement gqlJson)
{
    if (!gqlJson.TryGetProperty("data", out var data) ||
        !data.TryGetProperty("orders", out var orders))
        return "Não encontrei resultados.";

    var lines = new List<string> { "Pedidos encontrados:" };

    foreach (var o in orders.EnumerateArray())
    {
        var id = o.GetProperty("id").GetString();
        var name = o.GetProperty("customerName").GetString();
        var total = o.GetProperty("total").GetDecimal();
        var status = o.GetProperty("status").GetString();
        lines.Add($"- {id}: {name} — {status} — Total {total:C}");
    }

    return string.Join("\n", lines);
}

static string SummarizeOrder(JsonElement gqlJson, string id)
{
    if (!gqlJson.TryGetProperty("data", out var data) ||
        !data.TryGetProperty("orderById", out var order) ||
        order.ValueKind == JsonValueKind.Null)
        return $"Pedido {id} não encontrado.";

    var name = order.GetProperty("customerName").GetString();
    var total = order.GetProperty("total").GetDecimal();
    var status = order.GetProperty("status").GetString();
    return $"Pedido {id}: {name} — {status} — Total {total:C}";
}

public sealed record ChatRequest(string Text);
public sealed record ChatResponse(string CorrelationId, List<ChatMessage> Messages);
public sealed record ChatMessage(string Role, string Content);

```

## Discussão: Desafios e Oportunidades



## Interoperabilidade

Ao padronizar tools/resources/prompts, o MCP reduz o acoplamento entre host e serviços internos. Entretanto, a interoperabilidade real depende de contratos explícitos, versionamento, depreciação e documentação consistente (resources).

## Gestão de Dados Contextuais e Segurança

Em GraphQL, a flexibilidade é um benefício e um risco (SABBAG FILHO, 2025). Por isso, o desenho com allowlist e tools de domínio torna o acesso mais previsível. Além disso, a auditoria com CorrelationId permite rastrear decisões e execuções, condição essencial em ambientes regulados.

## Escalabilidade e Observabilidade

Ferramentas de observabilidade (logs estruturados, métricas e tracing) tornam-se fundamentais para identificar gargalos. Como agentes podem aumentar o volume e a variabilidade de chamadas, quotas e caches por operação são estratégias recomendáveis para manter previsibilidade.

## Conclusão

A construção de um MCP Server em .NET sobre um backend GraphQL permite combinar flexibilidade de consulta com governança rigorosa. Ao aplicar premissas claras, allowlist de operações, limites de acesso e auditoria, o MCP se torna uma camada eficaz para integrar agentes a sistemas corporativos de forma previsível. A API de demonstração em formato de chat ilustra a camada de apresentação, enquanto a separação entre tools e gateway preserva segurança e manutenibilidade.

## References

- AMERSHI, Saleema et al. Guidelines for human-AI interaction. In: Proceedings of the 2019 chi conference on human factors in computing systems. 2019. p. 1-13.
- KARRAS, Aristeidis et al. LLM-Driven Big Data Management Across Digital Governance, Marketing, and Accounting: A Spark-Orchestrated Framework. Algorithms, v. 18, n. 12, p. 791, 2025.
- MARJANOVIC, Rickard. Evaluating GraphQL over REST within an .NET Web API: A controlled experiment conducted by integrating with the Swedish Companies Registration Office. 2022.
- SABBAG FILHO, Nagib. Depth Control in GraphQL APIs with .NET: A Secure Approach to Mitigating DoS Attacks. Leaders Tec, v. 2, n. 23, 2025.



Nagib atua como Microsoft MVP, possui experiência como Líder de Arquitetura de Sistemas, Tech Manager, Professor Universitário e Desenvolvedor Sênior, com uma carreira marcada por sólidas conquistas acadêmicas e profissionais. É pós-graduado pelo SENAC e pela Universidade Presbiteriana Mackenzie, com MBA em Tecnologia de Software pela USP, além de ter participado de programas de extensão do MIT e da Universidade de Chicago. É autor de um artigo científico revisado por pares sobre chatbots, apresentado presencialmente na Universidade de Barcelona. Participa ativamente da comunidade técnica, com mais de 50 artigos publicados, incluindo mais de 10 artigos no iMasters. Como palestrante, participou de eventos de grande relevância nacional, como as trilhas de Arquitetura .NET do TDC São Paulo, Arquitetura de Soluções do TDC Floripa e Developer Technologies do MVP Conf, além de palestrar no TOTVS Developers.